

DATA STRUCTURES AND ALGORITHMS

LECTURE 14

Lect. PhD. Oneț-Marian Zsuzsanna

Babeș - Bolyai University
Computer Science and Mathematics Faculty

2025 - 2026

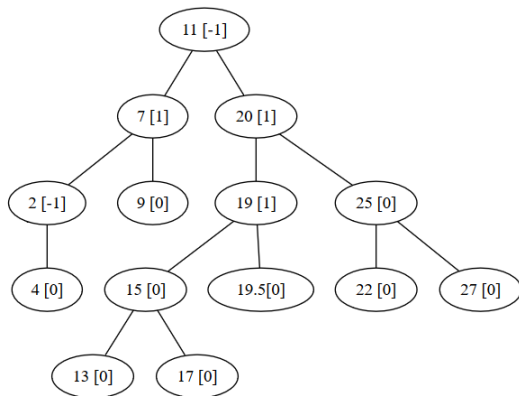
In Lecture 13...

- Binary Tree
- AVL Tree

- AVL Tree

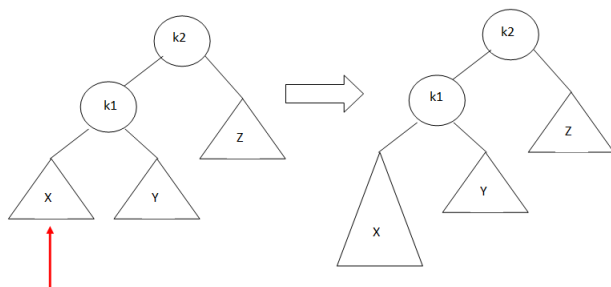
- Definition: An AVL (Adelson-Velskii Landis) tree is a binary tree which satisfies the following property (AVL tree property):
 - If x is a node of the AVL tree:
 - the difference between the height of the left and right subtree of x is 0, 1 or -1 (balancing information)
- Observations:
 - Height of an empty tree is -1
 - Height of a single node is 0

AVL Trees



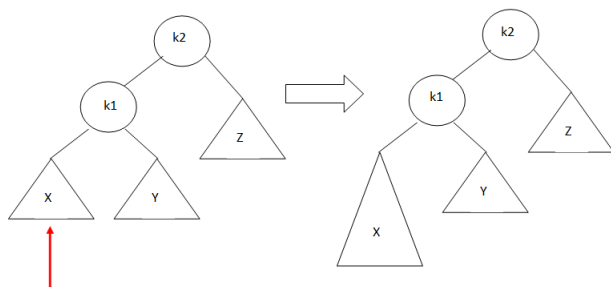
- This is an AVL tree.

AVL Trees - rotations - case 1



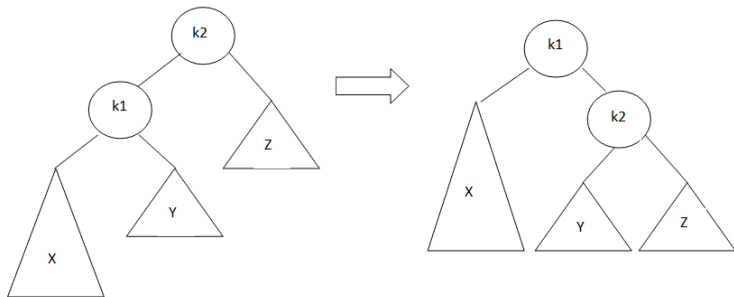
- Obs: X , Y and Z represent subtrees with the same height.

AVL Trees - rotations - case 1

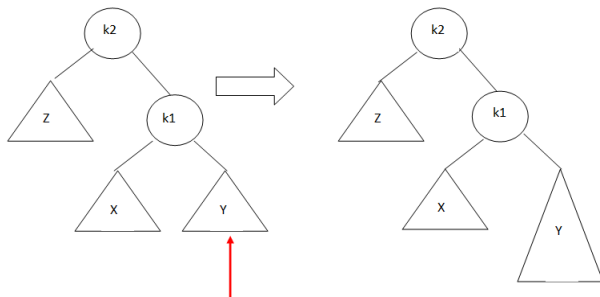


- Obs: X , Y and Z represent subtrees with the same height.
- Solution: single rotation to right

AVL Trees - rotation - Single Rotation to Right

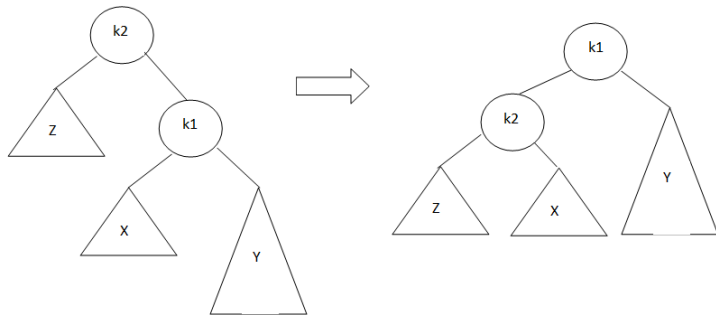


AVL Trees - rotations - case 4

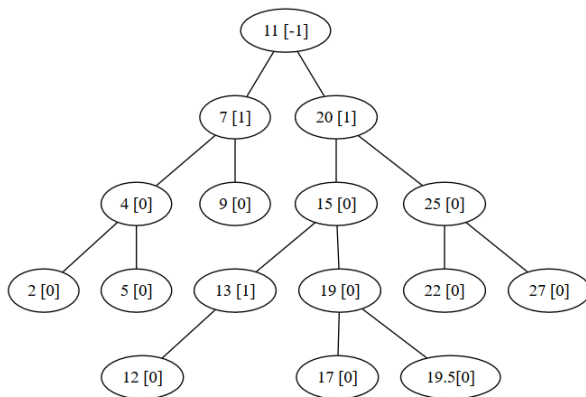


- Solution: **single rotation to left**

AVL Trees - rotation - Single Rotation to Left

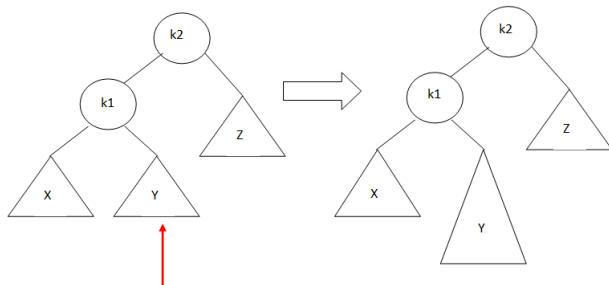


AVL Trees - rotation - case 4 example



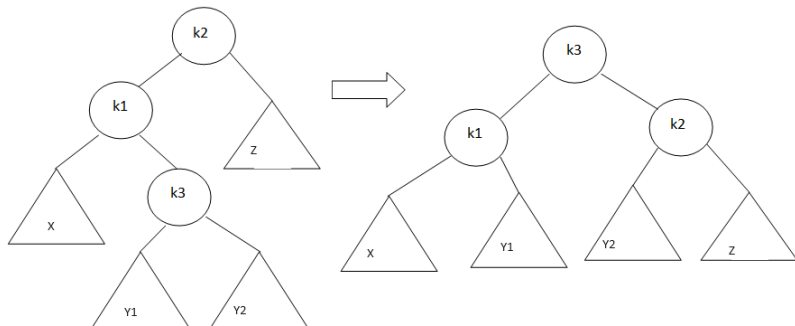
- After the rotation

AVL Trees - rotations - case 2

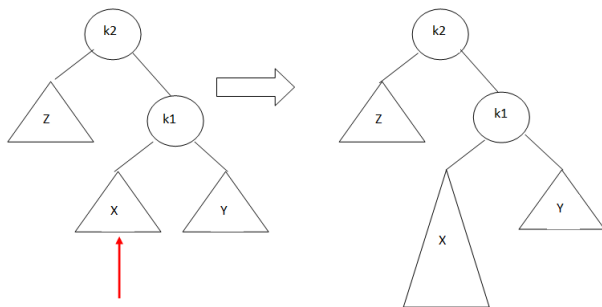


- Solution: **Double rotation to right**

AVL Trees - rotation - Double Rotation to Right

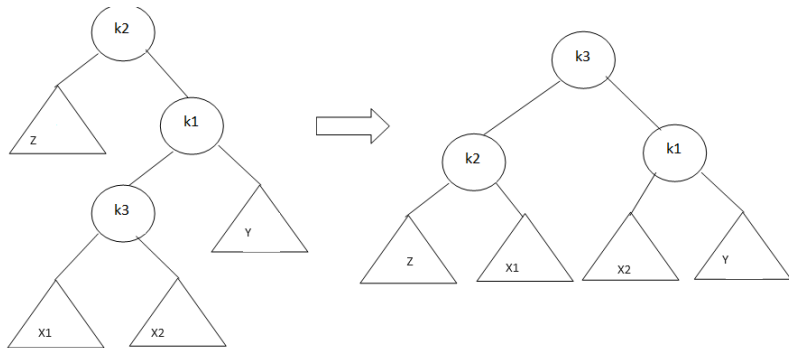


AVL Trees - rotations - case 3



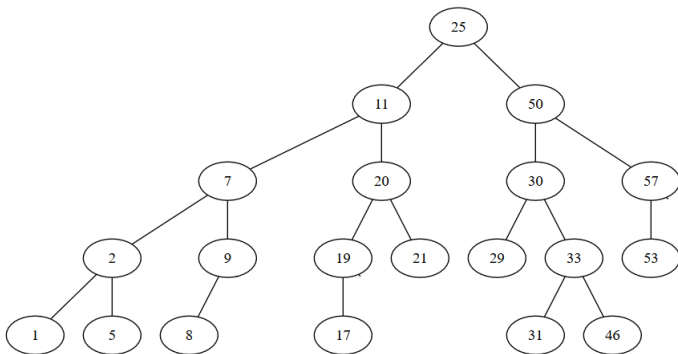
- Solution: **Double rotation to left**

AVL Trees - rotation - Double Rotation to Left



Example

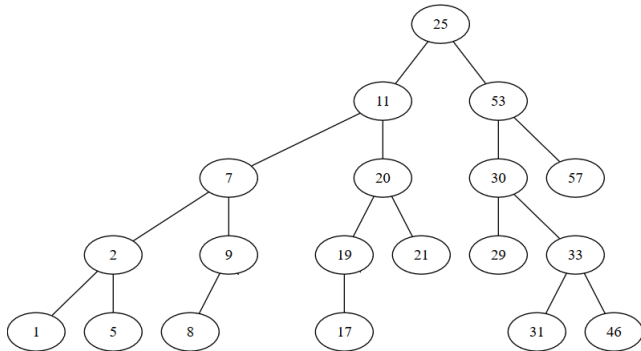
- After adding a lot of nodes, we arrived to this tree:



Example of remove I

- Remove 50

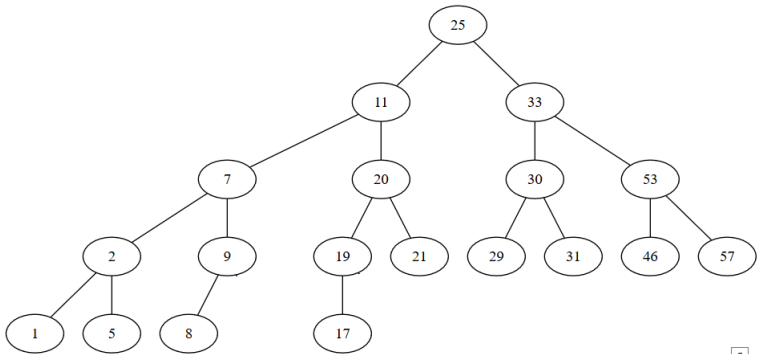
Example of remove II



- Do we need a rotation?
- If yes, on which node and what type of rotation?

Example of remove III

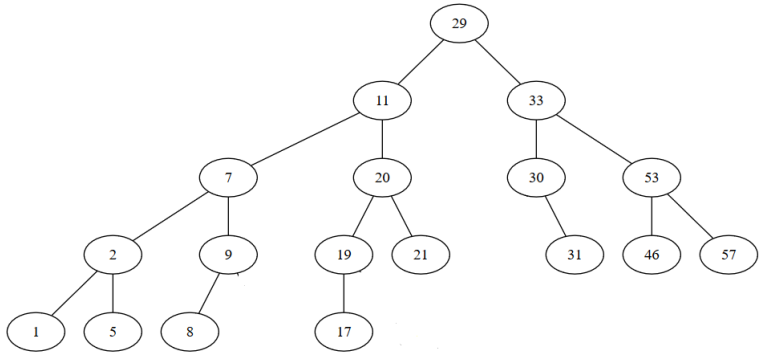
- Yes we need double right rotation on node 53
- After the rotation



G

- Remove 25

Example of remove IV

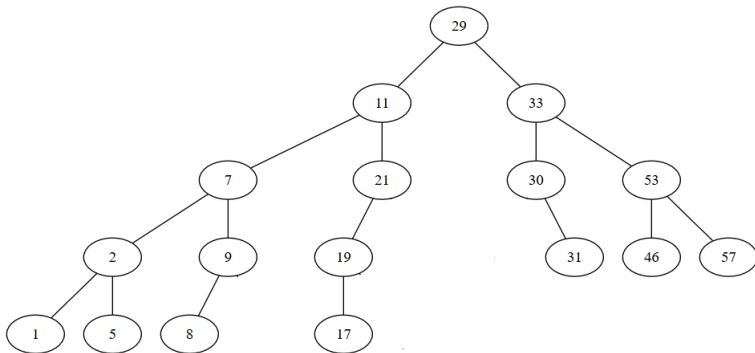


- Do we need a rotation?
- If yes, on which node and what type of rotation?

Example of remove V

- No rotation is needed
- Remove 20

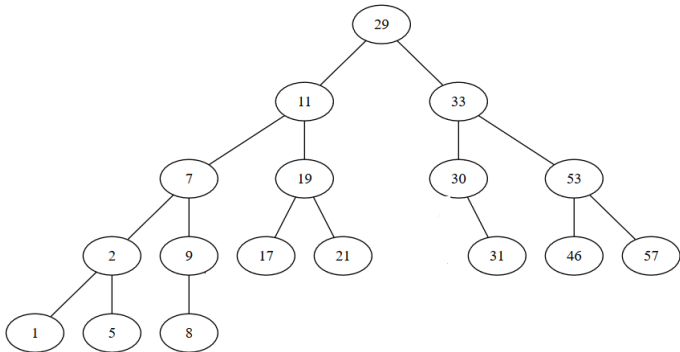
Example of remove VI



- Do we need a rotation?
- If yes, on which node and what type of rotation?

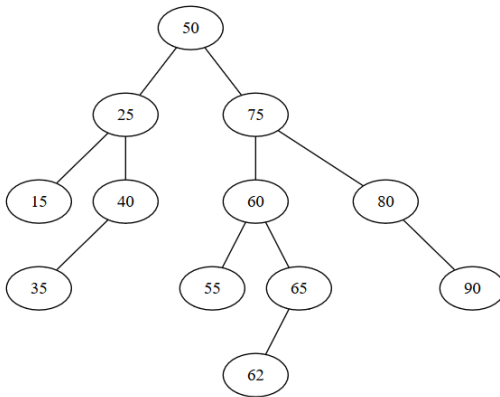
Example of remove VII

- Yes, we need a single right rotation on node 21
- After the rotation



Rotations for remove

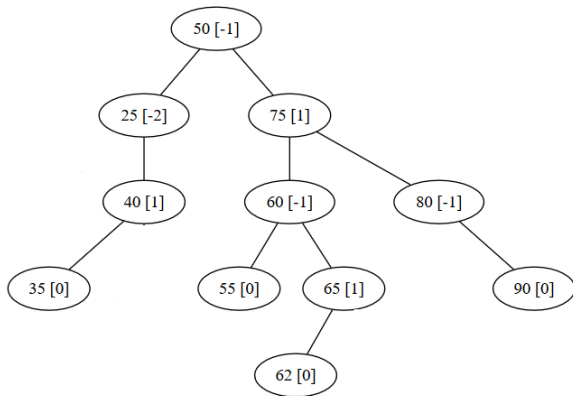
- When we remove a node, we might need more than 1 rotation:



- Remove value 15

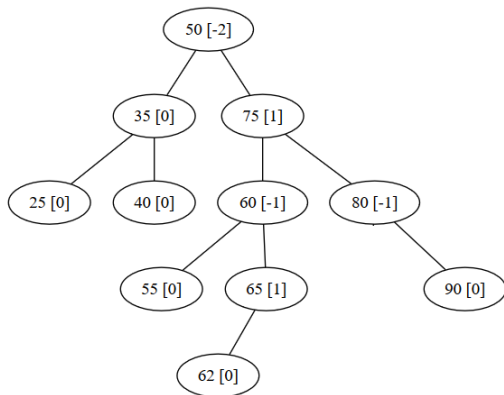
Rotations for remove

- After remove:



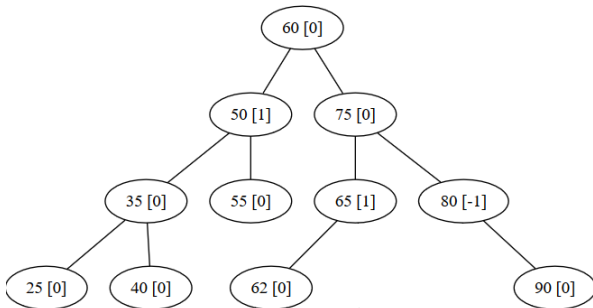
Rotations for remove

- After the rotation



Rotations for remove

- After the second rotation



AVL Trees - representation

- What structures do we need for an AVL Tree?

AVL Trees - representation

- What structures do we need for an AVL Tree?

AVLNode:

info: TComp *//information from the node*

left: $\uparrow$ AVLNode *//address of left child*

right: $\uparrow$ AVLNode *//address of right child*

h: Integer *//height of the node*

AVLTree:

root: $\uparrow$ AVLNode *//root of the tree*

- **Obs:** you might need to keep a relation in the AVLTree structure as well, we will consider the $\leq$ relation by default.

AVL Tree - implementation

- We will implement the *insert* operation for the AVL Tree.
- We need to implement some operations to make the implementation of *insert* simpler:

AVL Tree - implementation

- We will implement the *insert* operation for the AVL Tree.
- We need to implement some operations to make the implementation of *insert* simpler:
 - A subalgorithm that (re)computes the height of a node
 - A subalgorithm that computes the balance factor of a node
 - Four subalgorithms for the four rotation types (we will implement only one)
- And we will assume that we have a function, *createNode* that creates and returns a node containing a given information (left and right are NIL, height is 0).

AVL Tree - height of a node

subalgorithm `recomputeHeight(node)` **is:**

//pre: node is an $\uparrow$ AVLNode. All descendants of node have their height (h) set

//to the correct value

//post: if node $\neq$ NIL, h of node is set

AVL Tree - height of a node

subalgorithm `recomputeHeight(node)` **is:**

//pre: node is an $\uparrow$ AVLNode. All descendants of node have their height (h) set

//to the correct value

//post: if node $\neq$ NIL, h of node is set

if `node  $\neq$  NIL` **then**

if `[node].left = NIL and [node].right = NIL` **then**

`[node].h  $\leftarrow$  0`

else if `[node].left = NIL` **then**

`[node].h  $\leftarrow$  [[node].right].h + 1`

else if `[node].right = NIL` **then**

`[node].h  $\leftarrow$  [[node].left].h + 1`

else

`[node].h  $\leftarrow$  max ([[node].left].h, [[node].right].h) + 1`

end-if

end-if

end-subalgorithm

- Complexity: $\Theta(1)$

AVL Tree - balance factor of a node

function balanceFactor(node) **is:**

//pre: node is an $\uparrow$ AVLNode. All descendants of node have their height (h) set

//to the correct value

//post: returns the balance factor of the node

AVL Tree - balance factor of a node

function balanceFactor(node) **is:**

//pre: node is an $\uparrow$ AVLNode. All descendants of node have their height (h) set

//to the correct value

//post: returns the balance factor of the node

if [node].left = NIL **and** [node].right = NIL **then**

 balanceFactor $\leftarrow$ 0

else if [node].left = NIL **then**

 balanceFactor $\leftarrow$ -1 - [[node].right].h *//height of empty tree is -1*

else if [node].right = NIL **then**

 balanceFactor $\leftarrow$ [[node].left].h + 1

else

 balanceFactor $\leftarrow$ [[node].left].h - [[node].right].h

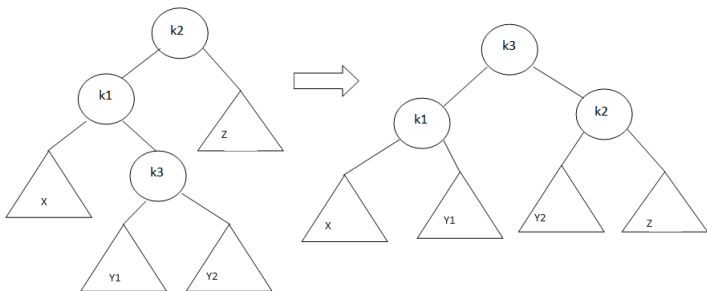
end-if

end-subalgorithm

- Complexity: $\Theta(1)$

AVL Tree - rotations

- Out of the four rotations, we will only implement one, double right rotation (DRR).
- The other three rotations can be implemented similarly (RLR, SRR, SLR).



function DRR(node) *is:* *//pre: node is an $\uparrow$ AVLNode on which we perform the double right rotation*

//post: DRR returns the new root after the rotation

k2 $\leftarrow$ node

k1 $\leftarrow$ [node].left

k3 $\leftarrow$ [k1].right

k3left $\leftarrow$ [k3].left

k3right $\leftarrow$ [k3].right

function DRR(node) *is:* *//pre: node is an $\uparrow$ AVLNode on which we perform the double right rotation*

//post: DRR returns the new root after the rotation

k2 $\leftarrow$ node

k1 $\leftarrow$ [node].left

k3 $\leftarrow$ [k1].right

k3left $\leftarrow$ [k3].left

k3right $\leftarrow$ [k3].right

//reset the links

newRoot $\leftarrow$ k3

[newRoot].left $\leftarrow$ k1

[newRoot].right $\leftarrow$ k2

[k1].right $\leftarrow$ k3left

[k2].left $\leftarrow$ k3right

//continued on the next slide

```
//recompute the heights of the modified nodes  
recomputeHeight(k1)  
recomputeHeight(k2)  
recomputeHeight(newRoot)  
DRR  $\leftarrow$  newRoot  
end-function
```

- Complexity: $\Theta(1)$

AVL Tree - insert

function insertRec(node, elem) **is**

//pre: node is a $\uparrow$ AVLNode, elem is the value we insert in the (sub)tree that

//has node as root

//post: insertRec returns the new root of the (sub)tree after the insertion

if node = NIL **then**

 insertRec $\leftarrow$ createNode(elem)

else if elem $\leq$ [node].info **then**

 [node].left $\leftarrow$ insertRec([node].left, elem)

else

 [node].right $\leftarrow$ insertRec([node].right, elem)

end-if

//continued on the next slide...

AVL Tree - insert

```
recomputeHeight(node)
balance  $\leftarrow$  getBalanceFactor(node)
if balance = -2 then
```

AVL Tree - insert

```
recomputeHeight(node)
balance  $\leftarrow$  getBalanceFactor(node)
if balance = -2 then
  //right subtree has larger height, we will need a rotation to the LEFT
  rightBalance  $\leftarrow$  getBalanceFactor([node].right)
  if rightBalance < 0 then
```

AVL Tree - insert

```
recomputeHeight(node)
balance ← getBalanceFactor(node)
if balance = -2 then
  //right subtree has larger height, we will need a rotation to the LEFT
  rightBalance ← getBalanceFactor([node].right)
  if rightBalance < 0 then
    //the right subtree of the right subtree has larger height, SRL
    node ← SRL(node)
  else
    node ← DRL(node)
  end-if
//continued on the next slide...
```

else if balance = 2 **then**

//left subtree has larger height, we will need a RIGHT rotation

leftBalance $\leftarrow$ getBalanceFactor([node].left)

if leftBalance > 0 **then**

AVL Tree - insert

```
else if balance = 2 then  
  //left subtree has larger height, we will need a RIGHT rotation  
  leftBalance  $\leftarrow$  getBalanceFactor([node].left)  
  if leftBalance > 0 then  
    //the left subtree of the left subtree has larger height, SRR  
    node  $\leftarrow$  SRR(node)  
  else  
    node  $\leftarrow$  DRR(node)  
  end-if  
end-if  
insertRec  $\leftarrow$  node  
end-function
```

- Complexity of the *insertRec* algorithm: $O(\log_2 n)$
- Since *insertRec* receives as parameter a pointer to a node, we need a wrapper function to do the first call on the root

subalgorithm insert(tree, elem) **is**

//pre: tree is an AVL Tree, elem is the element to be inserted

//post: elem was inserted to tree

tree.root $\leftarrow$ insertRec(tree.root, elem)

end-subalgorithm

- remove subalgorithm can be implemented similarly (start from the remove from BST and add the rotation part).

(Bad) examples

- During the semester we have talked about the most frequently used container ADTs and the data structures which are used in general to implement them.
- In the following, I am going to show you 3 real-life examples (and one artificial one), where knowing data structures and containers can help you write simpler / more efficient code.

Example 1

- Consider the following problem (simplified version of a problem given a few years ago for the Bachelor exam):
- *You have an ADT Student which has a name and a city. Write a function which takes as input a list of students and prints for each city all the students that are from that city. Each city should be printed only once and in any order.*
- How would you solve the problem? What container would you use?

Example 2

- Consider the following algorithm (written in Python and mentioned in our first Lecture as well):

```
def testContainer(container, l):  
    """  
    container is a container with integer numbers  
    l is a list with integer numbers  
    """  
    count = 0  
    for elem in l:  
        if elem in container:  
            count += 1  
    return count
```

- The above function counts how many elements from the list *l* can be found in the container. What is the complexity of *testContainer*?

Example 3

- Consider the following problem: *We want to model the content of a wallet, by using a list of integer numbers, in which every value denotes a bill. For example, a list with values [5, 1, 50, 1, 5] means that we have 62 RON in our wallet.*

Obviously, we are not allowed to have any numbers in our list, only numbers corresponding to actual bills (we cannot have a value of 8 in the list, because there is no 8 RON bill).

We need to implement a functionality to pay a given amount of sum and to receive rest 1f necessary.

There are many optimal algorithms for this, but we go for a very simple (and non-optimal): keep removing bills of the wallet until the sum of removed bills is greater than or equal to the sum you want to pay.

If we need to receive a rest, we will receive it in 1 RON bills.

Example 3

- For example, if the wallet contains the values [5, 1, 50, 1, 5] and we need to pay 43 RON, we might remove the first 3 bills (a total of 56) and receive the 13 RON rest in 13 bills of 1.

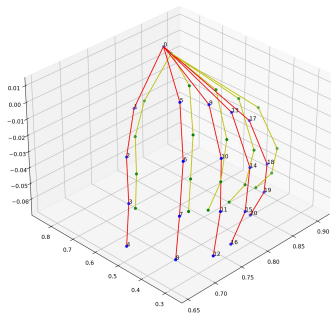
Example 3

- For example, if the wallet contains the values [5, 1, 50, 1, 5] and we need to pay 43 RON, we might remove the first 3 bills (a total of 56) and receive the 13 RON rest in 13 bills of 1.
- This is a Java implementation provided by a student. ArrayList is an implementation of ADT List using a dynamic array as data structure. What is wrong with it?

```
public void spendMoney(ArrayList<Integer> wallet, Integer amount) {  
    Integer spent = 0;  
    while (spent < amount) {  
        Integer bill = wallet.remove(0); //removes element from position 0  
        spent += bill;  
    }  
    Integer rest = spent - amount;  
    while (rest > 0) {  
        wallet.add(0, 1);  
        rest--;  
    }  
}
```

Example 4

- Consider an application which processes a video recording of a hand gesture (for ex. thumbs up).
- The figure on the right is a visualization of a hand for which 20 key points are identified by a library (actually there are two hands, one with red and one with green, but only the red one has the numbers displayed).
- That library actually provides the coordinates (x, y, z) for all 20 points.



Example 4

- The points are actually organized as a tree and the goal of the code below is to transform the coordinates of each key point (landmark in the code) to be relative to its parent.
- What is wrong with the code?

Example 4

```
def transform_to_relative_joints(frame):  
    # Defining hand hierarchy starting from wrist, ending with each finger's tip  
    parent_relationships = [  
        (0, 1), (1, 2), (2, 3), (3, 4), # Thumb  
        (0, 5), (5, 6), (6, 7), (7, 8), # Index finger  
        (0, 9), (9, 10), (10, 11), (11, 12), # Middle finger  
        (0, 13), (13, 14), (14, 15), (15, 16), # Ring finger  
        (0, 17), (17, 18), (18, 19), (19, 20)] # Little finger  
    relative_joints = []  
    # Processing each landmark of a frame  
    for i, landmark in enumerate(frame):  
        [x, y, z] = landmark  
        parent_index = None  
        # For a given landmark, search for its parent landmark/joint  
        for parent_relationship in parent_relationships:  
            if parent_relationship[1] == i:  
                parent_index = parent_relationship[0]  
                break  
  
        if parent_index is not None:  
            # When found, compute relative coordinates  
            ...  
            ...  
    return relative_joints
```

- Bachelor exam subject: recursive merge + complexity


```

vector<string> interclasare1(vector<string> v1, vector<string> v2) {
    if (v1.size() == 0) {
        return v2;
    }
    if (v2.size() == 0) {
        return v1;
    }
    vector<string> res;
    if (v1[0] < v2[0]) {
        res.push_back(v1[0]);
        v1.erase(v1.begin());
        auto v3 = interclasare1(v1, v2);
        res.insert(res.end(), v3.begin(), v3.end());
    }
    else {
        res.push_back(v2[0]);
        v2.erase(v2.begin());
        auto v3 = interclasare1(v1, v2);
        res.insert(res.end(), v3.begin(), v3.end());
    }
    return res;
}

```

```

vector<string> interclasare2(vector<string> v1, vector<string> v2) {
    if (v1.size() == 0) {
        return v2;
    }
    if (v2.size() == 0) {
        return v1;
    }

    vector<string> res;
    reverse(v1.begin(), v1.end());
    reverse(v2.begin(), v2.end());
    if (v1[v1.size()-1] < v2[v2.size()-1]) {
        res.push_back(v1[v1.size() - 1]);
        v1.pop_back();
    }
    else {
        res.push_back(v2[v2.size()-1]);
        v2.pop_back();
    }

    reverse(v1.begin(), v1.end());
    reverse(v2.begin(), v2.end());
    auto v3 = interclasare2(v1, v2);
    res.insert(res.end(), v3.begin(), v3.end());
    return res;
}

```

```

vector<string> interclasare3(vector<string> v1, vector<string> v2) {
    if (v1.size() == 0) {
        return v2;
    }
    if (v2.size() == 0) {
        return v1;
    }

    vector<string> res;
    string val;
    if (v1[v1.size() - 1] > v2[v2.size() - 1]) {
        val = v1[v1.size() - 1];
        v1.pop_back();
    }
    else {
        val = v2[v2.size() - 1];
        v2.pop_back();
    }
    res = interclasare3(v1, v2);
    res.push_back(val);
    return res;
}

```

Recap

- During the semester we have talked about the most important containers (ADT) and their main properties and operations
 - Bag, SortedBag, Set, SortedSet, Map, SortedMap, Multimap, SortedMultimap, List, SortedList, (Sparse)Matrix, Stack, Queue, Priority Queue and Deque, Binary Tree.
- We have also talked about the most important data structures that can be used to implement these containers
 - Dynamic array, Linked lists, Binary heap, Hash table (collision resolution with separate chaining, coalesced chaining, open addressing and linked hash table), Binary Search Tree, AVL Tree, Binary Tree.
- We have talked (briefly) about other data structures as well:
 - Skip list, Binomial heap, Cuckoo hashing, Perfect hashing.

Exam organization

- Without the required number of attendances, you cannot participate at the exam (new column in attendance sheet).
- Exam will be closed-book and will last for 100 minutes.

Exam organization

- Problems will be divided into 3 parts:
 - **PART A:** 4 multiple choice questions (0.4 points each)
 - **PART B:** 3 drawing questions (1 points each)
 - **PART C:** 1 problem, where either the complexity of a given piece of code needs to be computed, or some simple implementation needs to be done. (1.4 p) + 1 more complex implementation problem where you can choose between two problems with two different points (1.75p vs 3p)
- Solutions for each part need to be written on a different sheet of paper (so bring at least 3 sheets of paper) and your name and group needs to appear on all three of them.
- Justifications are important!

- Time management is important!
- Terminology is important. Example from previous year:
 - Questions: *What data structure would you use for ...?*
 - Answer (from an actual exam paper): *"I would use an ADT Sorted Bag data structure with key value pairs"*

- Today we have talked about:
 - AVL Trees
 - Bad examples
 - Exam